

Transformers Explained Step by Step

Summary:

This document explains Transformers step by step, using the toy example sentence:

“The cat sat on the mat.”

Transformers are the foundation of modern AI (translation, ChatGPT, summarization). They work by combining two key ideas:

1. **Embeddings** – turn words into vectors that capture meaning.
2. **Attention** – let words “look at” other words in the sentence to understand context.

The Full Pipeline

1. **Input & Embedding** – Words like “cat” and “dog” are placed in a vector space where similar words are close together.
2. **Positional Encoding** – Adds order information so the model knows word positions.
3. **Queries, Keys, Values (Q, K, V)** – Each word generates three signals: what it asks (Q), what it offers (K), what info it gives (V).
4. **Self-Attention** – Queries match with Keys to assign attention weights; words gather information from others’ Values.
 - Example: “cat” attends most to “sat” (action) and “mat” (location).
5. **Feedforward Network (FFN)** – Nonlinear layer that refines each word’s representation.
6. **Layer Normalization** – Balances values, keeping training stable.
7. **Encoding vs. Decoding** –
 - Encoder builds a full contextual map of the input.
 - Decoder generates output step by step (e.g., translation).

Key Takeaway

At the end of this process:

- “cat” knows it is the subject who sat on something.
- “sat” knows it’s the action of the cat.
- “mat” knows it’s the object being sat on.

This interaction of **embeddings + attention + layers** is what makes Transformers powerful for language, vision, and beyond.

The worked example: 'The cat sat on the mat', transformers are the backbone of modern AI (translation, ChatGPT, etc.).

Here's a simple, end-to-end explanation with concrete numbers and examples.

1. Input & Embedding

When we say a word is turned into an **embedding vector**, think of it as placing the word on a **map of meaning** with several axes (dimensions).

Here we're using **3 axes** to keep it simple.

Embedding Axes

- **Axis 1 (x)** → *Animal-ness vs. Object-ness*
 - Lower = animal-like, Higher = object-like
- **Axis 2 (y)** → *Domestic Pet-ness*
 - Higher = domestic pet (cat, dog, mat), Lower = not a pet (car, table)
- **Axis 3 (z)** → *Size / Scale*
 - Small values = small things (cat, mat), Larger values = big things (dog, car)

Example Word Embeddings

Word Embedding [Axis1, Axis2, Axis3]		Interpretation
cat	[0.20, 0.80, 0.10]	Small domestic animal (close to dog)
dog	[0.25, 0.85, 0.15]	Bigger domestic animal, similar to cat
car	[0.90, 0.10, 0.50]	Large object, far from cat/dog
mat	[0.15, 0.70, 0.20]	Small household object, often linked to cat/dog
sat	[0.30, 0.20, 0.40]	Action/verb (different cluster)
on	[0.10, 0.10, 0.30]	Relation word (small neutral vector)
the	[0.05, 0.05, 0.05]	Neutral filler, close to origin

How to Read This

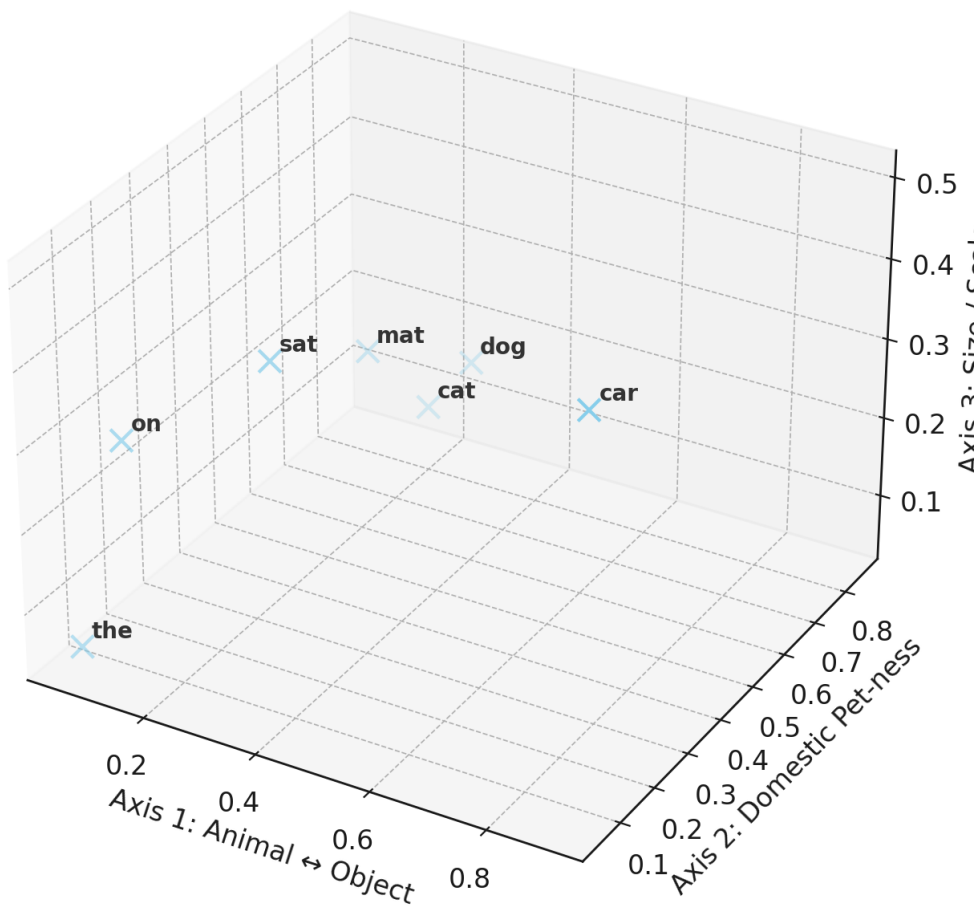
- **cat vs. dog** → close together (both animals, domestic pets, small-ish size).
- **cat vs. car** → far apart (animal vs. object).
- **cat vs. mat** → somewhat close (mat is domestic context, often linked to cat).
- **sat, on, the** → their embeddings are more “functional” and not tied to animal/object axes.

Visual Analogy

Imagine a **3D cube**:

- x-axis = *Animal-ness*
- y-axis = *Domestic Pet-ness*
- z-axis = *Size*
- “cat” and “dog” sit in the **same corner** (animal, domestic, small).
- “car” is far away (object, not domestic, large).
- “mat” is between pet + object (domestic but not alive).
- “sat/on/the” cluster near the center because they’re functional words.

3D Embedding Space Example 'The cat sat on the mat'



2. Positional Encoding

Transformers process all words **in parallel**, unlike RNNs (which process sequentially). This means the model has **no natural sense of order**. Without positional information, the sentence:

- “The cat sat on the mat”
would look the same as
- “Mat the sat cat the on”

Clearly, order matters! That’s why we add **positional encoding** to each word embedding.

How Positional Encoding Works

Transformers process all words **in parallel**, so by default they cannot tell **which word comes first, second, or last**.

Without positional encoding, the model would treat:

- “The cat sat on the mat”
and
- “Mat the sat cat the on”

as the **same bag of words**.

To solve this, we add a **positional vector** to each word embedding.

Step 1: Assign Each Position a Vector

Every position in the sequence (1st, 2nd, 3rd, ...) gets its own unique vector.

In our toy example (3D space):

Position Positional Vector (P)

1 (the)	[0.01, 0.02, 0.03]
2 (cat)	[0.05, 0.01, 0.02]
3 (sat)	[0.02, 0.03, 0.01]
4 (on)	[0.03, 0.01, 0.04]
5 (the)	[0.04, 0.02, 0.01]
6 (mat)	[0.06, 0.01, 0.02]

Step 2: Add Position Vector to Word Embedding

We combine **meaning** (from the word embedding) with **position** (from positional encoding).

Example with “cat” (**position 2**):

- Word embedding = [0.20, 0.80, 0.10]
- Position vector = [0.05, 0.01, 0.02]
- Final = [0.25, 0.81, 0.12]

Now the model knows both:

- The word is “cat” (meaning).
- It is the **2nd word** in the sentence (position).

Step 3: Why Add Instead of Concatenate?

- **Adding** keeps the dimensionality the same (still 3D here, or 512D/1024D in real models).
- This makes it efficient and lets the network smoothly learn position-aware meanings.
- Example: “cat” at the **start** of a sentence vs. “cat” at the **end** will have slightly different vectors, guiding the model to interpret them differently.

Analogy (Music Sheet)

- Word embeddings = the **notes** (C, D, E...).
- Positional encodings = the **timing** (1st beat, 2nd beat, etc.).
- Add them together → The model doesn’t just know *what* the note is, but also *when* to play it.

With positional encoding, “*The cat sat on the mat*” becomes a sequence where each word is uniquely identified by **both meaning and order**.

Example (Sentence = “The cat sat on the mat”):

Word	Base Embedding (E)	Positional Vector (P)	Final (E+P)
the(1)	[0.05, 0.05, 0.05]	[0.01, 0.02, 0.03]	[0.06, 0.07, 0.08]
cat(2)	[0.20, 0.80, 0.10]	[0.05, 0.01, 0.02]	[0.25, 0.81, 0.12]
sat(3)	[0.30, 0.20, 0.40]	[0.02, 0.03, 0.01]	[0.32, 0.23, 0.41]
on(4)	[0.10, 0.10, 0.30]	[0.03, 0.01, 0.04]	[0.13, 0.11, 0.34]
the(5)	[0.05, 0.05, 0.05]	[0.04, 0.02, 0.01]	[0.09, 0.07, 0.06]

Word	Base Embedding (E)	Positional Vector (P)	Final (E+P)
mat(6)	[0.15, 0.70, 0.20]	[0.06, 0.01, 0.02]	[0.21, 0.71, 0.22]

Interpretation

- The **word embedding** captures *what the word means*.
- The **positional encoding** captures *where it is in the sequence*.
- By adding them, the model can distinguish between:
 - “cat sat” (correct order) vs. “sat cat” (swapped).

How Are Positional Vectors Chosen?

In practice:

- The original Transformer paper (Vaswani et al., 2017) used **sine and cosine waves** to generate smooth, repeating position vectors.
- These allow the model to easily learn both **absolute positions** (word 2 vs word 5) and **relative positions** (word 2 is next to word 3).

In our toy example, we used **small fixed vectors** (like [0.05, 0.01, 0.02]) for simplicity.

With positional encoding added, each token in “*The cat sat on the mat*” now carries both its **meaning** and its **place in the sequence**, making it ready for Q, K, V transformations in the next step.

4. Self-Attention (Scores → Softmax → Weighted Values) Once each word has its **embedding + positional encoding**, the Transformer creates three new vectors for every word:

- **Query (Q):** What am I asking about?
- **Key (K):** What information do I contain?
- **Value (V):** What information should I give if someone attends to me?

This is done using learned weight matrices (**W_q**, **W_k**, **W_v**). For simplicity, in our toy example:

- Q = 2nd dimension (Domestic Pet-ness)
- K = 1st dimension (Animal/Object axis)
- V = 3rd dimension (Size/Scale)

Example with Sentence: “The cat sat on the mat”

Word	Final Embedding (E+P)	Q (2nd dim)	K (1st dim)	V (3rd dim)
the(1)	[0.06, 0.07, 0.08]	0.07	0.06	0.08
cat(2)	[0.25, 0.81, 0.12]	0.81	0.25	0.12
sat(3)	[0.32, 0.23, 0.41]	0.23	0.32	0.41
on(4)	[0.13, 0.11, 0.34]	0.11	0.13	0.34
the(5)	[0.09, 0.07, 0.06]	0.07	0.09	0.06
mat(6)	[0.21, 0.71, 0.22]	0.71	0.21	0.22

Interpretation in the Sentence

- **Cat’s Query (Q=0.81):**
“Who tells me what I’m doing, and where am I?”
- **Sat’s Key (K=0.32):**
“I contain an action.”
- **Mat’s Key (K=0.21):**
“I contain a place.”
- **Values:** These are the actual details they contribute once selected.
 - Sat’s Value = 0.41 (action info).
 - Mat’s Value = 0.22 (location info).

Analogy (Classroom Style)

Imagine a classroom:

- **Cat (student)** asks: “*What am I doing? Where am I?*” (Query).
- **Sat (student)** raises hand: “*I have action knowledge.*” (Key).
- **Mat (student)** raises hand: “*I have location knowledge.*” (Key).
- When called upon, they share their **Values**.
- Cat then updates its understanding based on what others contributed.

4. Self-Attention (Scores → Softmax → Weighted Values)

Self-attention is the **core mechanism** of Transformers. Each word looks at every other word, decides **how much attention to give**, and then builds a new representation of itself using those weighted contributions.

Step 1 – Compute Scores (Q·K)

For each word, we take its **Query (Q)** and dot it with all the **Keys (K)** of other words.

Example: **Cat (Q=0.81)** attends to all Keys:

- $\text{cat} \rightarrow \text{the}(1): 0.81 \times 0.06 = \mathbf{0.049}$
- $\text{cat} \rightarrow \text{cat}(2): 0.81 \times 0.25 = \mathbf{0.203}$
- $\text{cat} \rightarrow \text{sat}(3): 0.81 \times 0.32 = \mathbf{0.259}$
- $\text{cat} \rightarrow \text{on}(4): 0.81 \times 0.13 = \mathbf{0.105}$
- $\text{cat} \rightarrow \text{the}(5): 0.81 \times 0.09 = \mathbf{0.073}$
- $\text{cat} \rightarrow \text{mat}(6): 0.81 \times 0.21 = \mathbf{0.170}$

Raw scores (cat row) = [0.049, 0.203, 0.259, 0.105, 0.073, 0.170]

4.2 Apply Softmax

After computing raw scores ($\mathbf{Q \cdot K}$) for a word, we must **normalize** them so they become probabilities. This ensures:

1. All values are between 0 and 1.
2. They sum to 1.
3. Higher scores get more weight, but smaller scores are still considered.

Raw Scores for Cat (Q=0.81):

From Step 1, we had:

- $\text{cat} \rightarrow \text{the1} = 0.049$
- $\text{cat} \rightarrow \text{cat} = 0.203$
- $\text{cat} \rightarrow \text{sat} = 0.259$
- $\text{cat} \rightarrow \text{on} = 0.105$
- $\text{cat} \rightarrow \text{the2} = 0.073$
- $\text{cat} \rightarrow \text{mat} = 0.170$

Raw vector = [0.049, 0.203, 0.259, 0.105, 0.073, 0.170]

Step A: Exponentiate Each Score

Softmax uses exponentials to magnify differences.

$$e^x = \exp(x)$$

- $\exp(0.049) \approx \mathbf{1.050}$
- $\exp(0.203) \approx \mathbf{1.225}$
- $\exp(0.259) \approx \mathbf{1.296}$
- $\exp(0.105) \approx \mathbf{1.111}$
- $\exp(0.073) \approx \mathbf{1.076}$

- $\exp(0.170) \approx 1.185$

Step B: Normalize by Sum

$$\text{Sum} = 1.050 + 1.225 + 1.296 + 1.111 + 1.076 + 1.185 = \mathbf{6.943}$$

Now divide each $\exp(\text{score})$ by the total:

- $\text{the1} = 1.050 / 6.943 \approx \mathbf{0.151}$
- $\text{cat} = 1.225 / 6.943 \approx \mathbf{0.176}$
- $\text{sat} = 1.296 / 6.943 \approx \mathbf{0.187}$
- $\text{on} = 1.111 / 6.943 \approx \mathbf{0.160}$
- $\text{the2} = 1.076 / 6.943 \approx \mathbf{0.155}$
- $\text{mat} = 1.185 / 6.943 \approx \mathbf{0.171}$

Final Attention Distribution for Cat

Cat's Query assigns its attention as:

- **sat = 18.7%** (highest → the action)
- **mat = 17.1%** (second highest → the location)
- others (the, on, etc.) ~15–16%

Intuition

- Softmax **sharpens focus** → even though raw scores were close (0.259 vs. 0.170), after exponent + normalization, sat and mat clearly dominate.
- Without Softmax, the model wouldn't know how much weight to give each relationship.

Step 3 – Weighted Sum of Values (Context Vector)

Now each word's **new representation** is computed as the weighted sum of all **Values (V)**.

$$\text{Recall Values} = [0.08, 0.12, 0.41, 0.34, 0.06, 0.22]$$

For cat:

- $(0.151 \times 0.08) = 0.012$
- $(0.176 \times 0.12) = 0.021$
- $(\mathbf{0.187} \times \mathbf{0.41}) = 0.077$
- $(0.160 \times 0.34) = 0.054$
- $(0.155 \times 0.06) = 0.009$
- $(0.171 \times 0.22) = 0.038$

Sum = 0.21 (new enriched representation for “cat”).

Step 4 – Interpretation

- The cat’s updated embedding ≈ 0.21 , now infused with contextual meaning.
- Cat has learned:
 - Its **action** comes from *sat*.
 - Its **location** comes from *mat*.
- This is how Transformers build **context-aware word meanings**.

Analogy 🎤 (Classroom)

- Cat asks: “What am I doing? Where am I?”
- Sat raises hand: “I contain the action (sitting)!”
- Mat raises hand: “I contain the place (mat)!”
- Cat listens to both, weights their contributions (sat=18.7%, mat=17.1%), and builds a new understanding.

At the end of self-attention, **each word is updated with context from the entire sentence**.

- Sat knows who sat (cat).
- Mat knows what’s on it (cat).
- Cat knows what it did (sat) and where (mat).

5. Feedforward Network (FFN)

After self-attention, each token has a new **contextualized representation** (like “cat” = 0.21 after listening to “sat” and “mat”).

But attention alone is **linear** it’s just weighted averages.

To add **nonlinear transformations** and let the model capture richer patterns, we pass each token’s output through a **feedforward neural network (FFN)**.

Structure of the FFN

The FFN is applied **independently to each token**, but with the **same weights** across all tokens.

It has 3 steps:

1. **Linear Transformation #1**
 - Multiply by a weight matrix W_1 , add bias b_1 .

- Expands the dimensionality (e.g., from 3 $\rightarrow$ 12 hidden units).
- 2. **Nonlinear Activation**
 - Apply ReLU or GELU (squash negative values, keep positive patterns).
 - Introduces flexibility and nonlinearity.
- 3. **Linear Transformation #2**
 - Multiply by another weight matrix W_2 , add bias b_2 .
 - Compress back to original size (e.g., 12 $\rightarrow$ 3).

Toy Example: Cat's Representation

From attention: cat $\approx$ **0.21**

Let's say we use tiny numbers:

- $W_1 = 2, b_1 = 0$
- Activation = ReLU
- $W_2 = 0.5, b_2 = 0$

Step 1: Expand

$$h_1 = W_1 \times 0.21 = 0.42$$

Step 2: Nonlinear Activation

$$\text{ReLU}(0.42) = 0.42$$

Step 3: Compress

$$\text{Output} = W_2 \times 0.42 = 0.21$$

Here it looks unchanged because of our simple toy weights, but in practice:

- The FFN reshapes the information.
- It lets the model learn **abstract features** like “subject–verb agreement,” “tense consistency,” or “semantic roles.”

Analogy

- **Attention = Shopping list** (you gather ingredients from other tokens).
- **FFN = Cooking process** (you prepare the dish with recipes, spices, heat).
- Without the FFN, you'd just have raw ingredients; with it, you get a meal.

The FFN ensures that after attention, tokens are transformed into richer, nonlinear forms that capture higher-level relationships.

This makes the model powerful enough to learn language patterns beyond simple word associations

6. Layer Normalization

After **Self-Attention** and the **Feedforward Network (FFN)**, the outputs can vary wildly in scale.

To keep training stable and ensure the model learns efficiently, Transformers apply **Layer Normalization (LayerNorm)**

Why Normalize?

- Some tokens might produce very large values (e.g., verbs like “*sat*” that gather strong attention).
- Others may produce small values (like determiners “*the*”).
- Without normalization, these differences could destabilize learning.

LayerNorm ensures each token’s vector has:

- **Mean = 0**
- **Variance = 1**

This balances the contributions across the sequence.

The Formula

For a token vector $x = [x_1, x_2, \dots, x_d]$:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{(x - \mu)}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

Where:

- μ = mean of values in x
- σ^2 = variance of values in x
- γ, β = learned parameters (scale and shift)
- ϵ = tiny constant for stability

Toy Example (with Cat's Context Vector)

Suppose after FFN, **cat's representation** = [0.12, 0.21, 0.34]

1. Compute Mean

$$\mu = \frac{0.12 + 0.21 + 0.34}{3} = 0.22$$

2. Compute Variance

$$\sigma^2 = \frac{(0.12 - 0.22)^2 + (0.21 - 0.22)^2 + (0.34 - 0.22)^2}{3}$$

$$= (0.01^2 + 0.01^2 + 0.12^2)/3 = 0.014 / 3 \approx 0.0047$$

3. Normalize Each Value

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

- $(0.12 - 0.22) / \sqrt{0.0047} \approx -1.45$
- $(0.21 - 0.22) / \sqrt{0.0047} \approx -0.15$
- $(0.34 - 0.22) / \sqrt{0.0047} \approx +1.60$

4. Apply Scale (γ) and Shift (β)

If $\gamma=1$ and $\beta=0$ (for simplicity):

Final normalized vector = [-1.45, -0.15, +1.60]

Interpretation

- The values are now centered and balanced.
- No single number dominates; each dimension contributes fairly.
- During training, γ and β allow the model to **re-scale** and **shift** values as needed.

Analogy

Think of normalization like **meditation** before the next lesson:

- You calm down extreme highs and lows (values).
- You bring balance so learning continues smoothly.

With LayerNorm applied, the Transformer can stack many layers (Attention → FFN → Norm → repeat) without instability.

7. Encoding vs. Decoding

Transformers are made of **two main parts**:

1. **Encoder** → Understands the input.
2. **Decoder** → Generates the output (like a translation).

Both are built from the same blocks: **Embeddings** → **Attention** → **FFN** → **LayerNorm**, but they are used differently.

7.1 Encoder

The encoder takes the full input sentence at once:

“The cat sat on the mat”

- **Step 1: Embedding + Positional Encoding**
Each word gets its meaning + position.
- **Step 2: Self-Attention**
Every word looks at every other word.
 - Cat attends most to **sat** (action) and **mat** (location).
 - Sat attends to **cat** (subject).
 - Mat attends to **cat** and **sat**.
- **Step 3: FFN + LayerNorm**
Each token's vector is refined and normalized.

After several encoder layers, we get **contextual embeddings**:

- **Cat** → subject doing something.
- **Sat** → action of cat.
- **Mat** → location of action.

The encoder's output is a **semantic map** of the input sentence.

7.2 Decoder

The decoder generates the output **step by step**, using two types of attention:

1. **Masked Self-Attention**

- The decoder can only look at **previously generated words**, not future ones.
 - Example: when predicting the 3rd word, it only sees words 1 and 2.
2. **Encoder–Decoder Attention**
- The decoder looks back at the encoder’s outputs.
 - This lets it ground the output in the meaning of the input.

7.3 Example: Translation

Input:

“The cat sat on the mat.”

Decoder generates French translation word by word:

1. Starts with special <BOS> token.
2. Predicts “Le” (pays attention to “the cat”).
3. Predicts “chat” (strong attention to “cat”).
4. Predicts “s’est assis” (attention on “sat”).
5. Predicts “sur” (attention on “on”).
6. Predicts “le tapis” (attention on “the mat”).

Final output:

“Le chat s’est assis sur le tapis.”

7.4 Analogy

- **Encoder = Reader** → Reads the entire story and builds understanding.
- **Decoder = Storyteller** → Retells the story in another language, step by step, always checking back with the reader’s notes.=

Key Difference

- **Encoder:** processes the input **in parallel** (all words at once).
- **Decoder:** generates the output **sequentially** (one word at a time), using both **past outputs** and the **encoder’s context**.

With Encoding and Decoding combined, the Transformer can handle powerful tasks like **translation, summarization, code generation, and dialogue (ChatGPT itself!)**.

Transformer Pipeline with Example Vectors & Attention Weights
Sentence: 'The cat sat on the mat'

